

PHASIS — lista de comandos

Como rodar o programa, e que teste cada comando permite fazer

commit 982f376 · 22 de agosto de 2026

Legenda: ▷ o que o comando permite testar · ▶ armadilha. Todos os caminhos são relativos à raiz do repositório, exceto onde o comando começa com `cd dipole`.

1. Ambiente

Só o `dipole` com `--use-F3 1` e os geradores em Python precisam de biblioteca externa. O PHASIS compila e roda com `g++` e a STL, sem mais nada.

```
export PATH=/home/rafael/micromamba/envs/dis/bin:$PATH
export LD_LIBRARY_PATH=/home/rafael/micromamba/envs/dis/lib:$LD_LIBRARY_PATH
```

▷ LHAPDF 6.5.6, yadism 0.13.11 e os conjuntos CT10nlo e NNPDF31_nlo_as_0118 vivem nesse env micromamba, não no Python do sistema.

```
cd dipole && make env
```

▷ Diz se o LHAPDF foi detectado. Sem as duas linhas acima ele reporta ausente e um binário já compilado falha com `libLHAPDF.so: cannot open shared object file`.

2. Compilar

```
make -j8
```

▷ Constrói `trace`, `alpha_scan`, `redshift_scan` e as oito fases de teste. Também há `CMakeLists.txt`, mas o Makefile não exige `cmake` no PATH.

```
cd dipole && make -j8 && make build/sigma_nuN -j8
```

▷ O `dipole` tem build próprio. O acoplamento com o PHASIS é *por arquivo* — as tabelas de seção de choque — e não por link, então os dois nunca precisam compilar juntos.

3. Suíte de aceitação

```
make test
```

▷ 247 verificações em 8 fases, ~75 s no total. Deve terminar com 0 falhas em todas. É o primeiro comando a rodar depois de qualquer mudança.

```
./build/test_phase1      # 0,02 s -- quadratura, unidades, casos analiticos
./build/test_phase2      # 0,03 s -- relatividade geral, ponto de retorno, deflexo
./build/test_phase3      # 10 s   -- perfil com theta, rota de EDO acoplada
./build/test_phase4      # 6 s    -- cascata NC analitica, kernel, round-trip
./build/test_phase5      # 0,04 s -- casca tau = 1, protocolo de razes
./build/test_phase6      # 0,35 s -- topologia do raio emitido, cone de escape
./build/test_phase7      # 0,06 s -- acoplamento GR x saturao, alpha_eff
./build/test_phase8      # 58 s   -- corrente neutra de ponta a ponta
```

▷ Rodar isolado quando se quer o detalhe. Cada um imprime linha a linha o valor obtido, a referência e o desvio relativo.

▶ As fases 7 e 8 leem as tabelas em `dipole/data/`. Se elas não existirem, o teste *falha* com a mensagem de como gerá-las — não pula em silêncio.

```
cd dipole && make check
```

▷ Roda test-quadrature, test-table e validate-hera. O primeiro compara $|\psi_T|^2$ e $|\psi_L|^2$ contra Kutak–Kwieciński eqs. (12) e (13); deve dar desvio 0,00%.

4. Gerar as tabelas do dipolo

4.1. Corrente carregada

```
cd dipole
./build/sigma_nuN --model GBW --current CC --NE 300 --logEmin 3 --logEmax 14 \
  --nodesQ 16 --nodesX 16 --Ntabx 121 --NtabQ 121 \
  --use-F3 1 --pdf-set NNPDF31_nlo_as_0118
```

▷ ~50 s. Produz data/sigma_nuN_CC_GBW.dat com as nove chaves de metadados e o hash do commit. Trocar --model GBW por IIM dá o bCGC.

4.2. Corrente neutra, total e diferencial

```
./build/sigma_nuN --model GBW --current NC --NE 300 --logEmin 3 --logEmax 14 \
  --nodesQ 16 --nodesX 16 --nY 120 \
  --use-F3 1 --pdf-set NNPDF31_nlo_as_0118
```

▷ ~1,5 min (o NC tem 4 canais de dipolo contra 2 do CC). Produz sigma_nuN_NC_GBW.dat e dsigma_dy_NC_GBW.dat. O cabeçalho do segundo reporta, por energia, que fração de σ a integral em y recupera — deve ficar entre 0,999 e 1,011.

► O teste de física que vale fazer: σ_{NC}/σ_{CC} tem de convergir para $\approx 0,42$. Foi ele, e não nada interno ao código, que pegou uma tabela NC com os acoplamentos errados que dava 2,49.

```
python3 -c "
import numpy as np
cc=np.loadtxt('data/sigma_nuN_CC_GBW.dat'); nc=np.loadtxt('data/sigma_nuN_NC_GBW.dat')
E,r=cc[:,0],nc[:,2]/cc[:,2]
for v in (1e5,1e9,1e13):
    k=np.argmin(abs(E-v)); print('E=%9.2g NC/CC=%.4f'%(E[k],r[k]))
print('previsto: (1.3127/4)*(M_Z/M_W)^2 = 0.4224')"
```

▷ Deve dar 0,358 / 0,405 / 0,412. A subida com a energia é o $x F_3$ do CC saindo de cena — ele é de valência.

4.3. Convergência da quadratura

```
for n in 16 48 128; do
  ./build/sigma_nuN --model GBW --current CC --NE 80 --logEmin 7 --logEmax 14 \
    --use-F3 0 --nodesQ $n --nodesX $n --quiet
  cp data/sigma_nuN_CC_GBW.dat /tmp/nos_$n.dat
done
```

▷ As três saídas devem concordar em $\sim 5 \times 10^{-7}$. Se divergirem em 10^{-3} e a diferença cair como N^{-1} em vez de N^{-4} , há descontinuidade num extremo de quadratura — foi exatamente esse o sintoma do bug do extremo $y = 1$, que custava um fator 670 em ruído.

```
./build/sigma_nuN --model GBW --current CC --NE 80 --logEmin 7 --logEmax 14 \
  --use-F3 0 --Ntabx 241 --NtabQ 241 --quiet
```

▷ Dobrar a tabela de F não deve mudar *nada*. Se mudar, o erro está na interpolação; se não mudar mas o resultado variar com --nodesQ, está na quadratura de σ . Separar as duas causas é o que o par de comandos permite.

5. Validar o dipolo contra dados

```
cd dipole
./build/validate_hera --model GBW --Nr 400 --Nz 200 \
  --Q2min 0.25 --Q2max 50 --xmax 0.01 --flavors udsc --mq 0.14
```

► Limite eletromagnético contra 239 pontos de H1+ZEUS combinados, EPJ C75 (2015) 580. Deve dar $\langle \text{modelo/dado} \rangle = 1,046$. Escreve data/hera_validation.dat com pull por ponto.

```
./build/validate_hera --model IIM --iim-x0 0.69e-3 --Nr 400 --Nz 200
```

► O bCGC. Vale variar $--iim-x0$: com $x_0 = 0,00069 \times 10^{-6}$ — o valor que estava no código, com o expoente da *incerteza* lido como multiplicador — a razão cai para 0,10 contra o próprio dado a que o modelo foi ajustado.

```
./build/dipole --mode wavefunctions --current NC --flavor u --Q2 10
./build/dipole --mode integrand --current CC --flavor u --x 1e-5 --Q2 10
./build/dipole --mode sigma --current CC --flavor u --x 1e-5 --Q2 10
```

► Diagnóstico ponto a ponto. O modo *integrand* é o que mostra o pico em $r \sim 1/\varepsilon = 0,025 \text{ GeV}^{-1}$ em $Q^2 = M_W^2$ — que uma grade uniforme em r perde inteiro.

6. A tabela colinear

Serve de referência: as mesmas convenções e a mesma fórmula mestra do dipolo, com funções de estrutura colineares em vez do modelo de dipolo.

```
python3 tools/build_collinear_sf.py
```

► ~13 min. Tabula F_2 , F_L e xF_3 com yadism NLO sobre NNPDF3.1 numa grade 121×71 em (x, Q^2) .

► Fatia por Q^2 e grava cada fatia, então o RSS fica constante em ~800 MB e uma interrupção custa uma fatia. Pedir os 8591 pontos de uma vez custaria 8,4 GB e travaria a máquina — já aconteceu. Rodar de novo retoma de onde parou.

```
python3 tools/make_collinear_table.py --NE 300
```

► ~5 s. Integra com a *mesma* fórmula mestra do C++. Uma quarta coluna diz que fração de σ vem de x abaixo do grid do PDF: 0% até $2 \times 10^{10} \text{ GeV}$, 21% em $1,3 \times 10^{13}$. Acima disso α_{eff} colinear é propriedade da extrapolação, não medida.

► Confere com a literatura: $\sigma_{CC}(10^6 \text{ GeV}) \approx 6 \times 10^{-34} \text{ cm}^2$. Se sair fora disso por mais de 20%, o cartão de teoria ou o conjunto de PDF mudou.

7. Medir o acoplamento GR $\times$ saturação

```
./build/alpha_scan --dipole dipole/data/sigma_nuN_CC_GBW.dat \
  --colinear data/sigma_nuN_CC_collinear.dat \
  --out data/alpha_scan_GBW.csv --nE 240
```

► Tabula $\alpha_{\text{eff}} = d \ln \sigma / d \ln E$ e o teto $3^{\alpha/2}$ para as duas tabelas. O teto tem de ser *não-crescente* em $E \geq 10^7 \text{ GeV}$ — é a assinatura da saturação, e é ela que distingue o efeito de um mero reajuste de PDF.

► Chama `assert_comparable` antes de qualquer conta. Passar uma tabela de ν e uma de $\bar{\nu}$, ou uma NC e uma CC, faz lançar. Vale tentar de propósito, para ver o guarda funcionar: `--colinear dipole/data/sigma_nuN_NC_GBW.dat`.

```
./build/redshift_scan --dipole dipole/data/sigma_nuN_CC_GBW.dat \
--colinear data/sigma_nuN_CC_collinear.dat \
--out data/redshift_scan_GBW.csv --nE 9
```

► Mede $R(b, E) = (\tau_{\text{full}} - \tau_{\text{frozen}}) / \tau_{\text{frozen}}$ com a *mesma* geometria; só o argumento de σ muda. R é invariante sob $\sigma \rightarrow k\sigma$, então isola a forma e não a normalização.

► Geometria configurável: `--rs --rho0 --r0 --p --rin --rout`. Vale testar `--rin 2.0`: com r_{in} acima da esfera de fótons ($1,5 r_s$), os raios quase-críticos enrolam no vazio e o teto $3^{a/2}$ deixa de ser atingível. O default 1,2 é deliberado.

8. Traçar raios

```
./build/trace
```

► Sem argumento, imprime todas as chaves de configuração reconhecidas. É a documentação canônica do formato.

```
./build/trace data/exemplo_terra.cfg saida_terra.csv
./build/trace data/exemplo_disco.cfg saida_disco.csv
./build/trace data/exemplo_buraco_negro.cfg saida_bn.csv
```

► Três casos prontos. O CSV traz 23 colunas: `E_inf_GeV`, `b_cm`, `i_rad`, `psi_t_rad`, `r_min_cm`, `tau`, `P_surv`, `tau_inbound`, `tau_outbound`, `column_g_cm2`, `path_cm`, `theta_min`, `theta_max`, `n_disk_crossings`, `crosses_matter`, `captured`, `near_critical`, `deflection_rad`, `E_loc_max_GeV`, `winding_turns`, `tolerance_met`, `error`, `error_msg`. Os dois ramos aparecem separados — `tau_inbound` e `tau_outbound` — e só coincidem em perfil esférico.

► `exemplo_buraco_negro.cfg` é caso de *estresse numérico*, não cenário físico: põe matéria na esfera de fótons, o que não tem órbita estável. Está no cabeçalho do próprio arquivo.

```
# raio quase-critico: b logo acima de b_crit = 2.5980762 r_s
cat > /tmp/critico.cfg <<'EOF'
metric = schwarzschild
r_s_cm = 1.0e6
density = powerlaw_halo
rho0_g_cm3 = 3.0
r0_cm = 1.0e8
p = 2.0
r_in_cm = 1.2e6
r_out_cm = 1.0e10
xsec = powerlaw
sigma0_cm2 = 1e-33
E0_GeV = 1e6
alpha = 0.36
E_inf_GeV = 1e8
b_min_cm = 2.5980763e6
b_max_cm = 3.0e6
n_b = 20
deflection = 1
EOF
./build/trace /tmp/critico.cfg /tmp/critico.csv
```

► Varre o regime quase-crítico. τ vai de 199 a 1384 — diverge *logaritmicamente* quando $b \rightarrow b_{\text{crit}}^+$, e isso é físico: o raio enrola na esfera de fótons e o comprimento próprio cresce mesmo. A coluna `r_min_cm` tende a $1,5 r_s = 1,5 \times 10^6$.

► O resumo reporta degradados: 7 de 20 — sete raios não atingiram a tolerância. Não é falha silenciosa: a coluna `tolerance_met` marca quais. Apertar com `rel_tol = 1e-10` ou aumentar `max_evals` resolve, ao custo de tempo. Um resultado que não atingiu a tolerância pedida é sempre sinalizado, nunca entregue como se tivesse atingido.

```
# no mesmo arquivo, trocar as quatro linhas de xsec por:
#   xsec      = table
#   table_path = dipole/data/sigma_nuN_CC_GBW.dat
#   ./build/trace /tmp/critico.cfg /tmp/critico_tabela.csv
```

► Troca a lei de potência pela tabela real. Com `xsec_current = total` é *obrigatório* dar `table_path_nc` ou `nc_to_cc_ratio`: não há razão NC/CC assumida por default, porque as tabelas do dipole são CC puras.

9. Plots

```
python3 plots/literature.py
```

► Quatro figuras em `plots/`, PDF vetorial e PNG: $\sigma(E)$ contra Gandhi et al. (via Formaggio & Zeller, RMP 84 (2012) 1307, eqs. 90–91), α_{eff} e o teto, σ_{NC}/σ_{CC} , e o HERA.

► As curvas da referência são desenhadas só na faixa em que ela se declara válida (10^{16} a 10^{21} eV). Estendê-las seria atribuir-lhe uma afirmação que não faz, e justo onde a saturação muda a resposta.

```
cd dipole && make plot-sigma-nuN plot-scan-x plot-hera-validation
```

► Alvos de plot do dipole. `make` sozinho lista os disponíveis no início do arquivo.

10. O que ainda não funciona

```
# NAO faça isto esperando resultado correto:
#   raio EMITIDO a distancia finita + perfil nao esferico
#   (density = flared_thin_disk, ou opts.force_ode)
```

► A rota de EDO nunca recebeu a topologia de emissão. Para raio emitido ela integra os dois ramos como se viesse do infinito: erra por fator 7 num caso, lança exceção noutro, e devolve $\tau = 0$ em silêncio no caso radial. Perfil esférico com raio vindo do infinito está validado; o resto não. Idem para `transport_cascade`.

```
python3 -c "
import sys; sys.path.insert(0, 'dipole/tests/oracle')
import dipole_oracle as o
print('oraculo %.4e cm2' % o.sigma_cm2(1e9, NlogQ=8, Nlogx=8, Nr=100, Nz=100))"

cd dipole && ./build/sigma_nuN --model GBW --current CC \
  --NE 1 --logEmin 9 --logEmax 9 --use-F3 0 --quiet
```

► Dá $7,628 \times 10^{-33}$ contra $7,352 \times 10^{-33}$ — 3,8% de diferença. Mas a comparação hoje não isola nada, porque o oráculo está atrás de *duas* mudanças estruturais: a regra de densidade de nós por década (a quadratura dele ainda usa número fixo) e o conserto do extremo $y = 1$. E não conhece NC. Ele existe para provar que uma mudança no C++ fez o que se pretendia; enquanto estiver divergente, não serve para isso. É o problema **P4** do relatório de estado.